

Progetto HorseCare

Dall'Architettura al Rilascio su Kubernetes e Cloudflare

Antonio Sgalla

Università degli Studi di Camerino

June 22, 2026

Indice dei Contenuti

- 1 Infrastruttura di Virtualizzazione (On-Premise)
- 2 Architettura e Stack Tecnologico
- 3 Containerizzazione e Sviluppo Locale
- 4 Deployment in Kubernetes (K8s)
- 5 Esposizione e Pubblicazione

Infrastruttura Cloud Privata: Proxmox VE e Debian 13

L'intero cluster Kubernetes è ospitato su un'infrastruttura di virtualizzazione dedicata (On-Premise).

- **Host Bare-Metal:** Server fisico basato sul software di virtualizzazione open-source **Proxmox VE (PVE)**.
- **Cluster K8s a 3 Nodi:** Tre macchine virtuali distinte (node-1, node-2, node-3) rispettivamente master, worker e worker, configurate per ripartire il carico di lavoro:
 - Distribuzione Linux di base uniforme: **Debian 13 (Trixie)**.
 - Risorse allocate per nodo per garantire la fluidità di PostgreSQL e Longhorn.
- **Automazione del Provisioning:** L'inizializzazione del sistema operativo, la gestione dei pacchetti e la configurazione dei prerequisiti di rete sono state interamente automatizzate tramite **script bash personalizzati** (`common.sh`).

Rilevanza Architettuale

L'architettura a 3 nodi attuale non garantisce HA, ma ampliando questo setup è possibile garantirla.

Stack Tecnologico (API-First)

- **Data Layer:** PostgreSQL 16+ per memorizzazione sicura e integrità referenziale.
- **Backend API:** PostgREST per la generazione dinamica di REST API direttamente dal database.
- **Documentazione:** Swagger UI integrato per il testing in tempo reale.
- **Presentation Layer:** Angular 19+ (Standalone Components, RxJS, TypeScript).
- **Infrastruttura locale:** Nginx come Reverse Proxy e Docker Compose per lo sviluppo locale.

Scelta Progettuale

L'architettura API-first disaccoppia completamente lo strato di presentazione (client) da quello dei dati, abilitando scalabilità e sviluppo parallelo.

- **Schema Relazionale:** Gestione tabelle utenti ('auth.users'), scuderie ('stables'), cavalli ('horses'), ed eventi pianificati ('events').
- **Identificativi UUID:** Utilizzo di UUID per garantire l'unicità ed evitare conflitti in caso di future migrazioni.
- **Row Level Security (RLS):** I dati sono protetti a livello di database; gli utenti vedono solo le scuderie e i cavalli associati al proprio ID.
- **Automazione:** Trigger PL/pgSQL che aggiornano automaticamente i campi di tracciamento temporale ('updated_at').

PostgREST

- Legge direttamente il catalogo del database.
- Traduce le richieste HTTP in interrogazioni SQL altamente ottimizzate.
- Gestisce l'autenticazione tramite token JWT emessi durante il login/signup.

Swagger UI

- Genera interattivamente la documentazione dell'API leggendo le specifiche OpenAPI da PostgREST.
- Consente agli sviluppatori di testare in sicurezza le query direttamente da browser.

Presentation Layer: Angular

- **Componenti Standalone:** Struttura dell'applicazione Angular snella, priva di moduli pesanti.
- **Design Glassmorphism:** Interfaccia elegante con effetti sfocati ('backdrop-filter'), ombreggiature morbide e font premium.
- **Mobile-Adaptive:** Progettato con una Bottom Navigation Bar su schermi piccoli (target mobile-first per uso in scuderia) e sidebar classica su schermi desktop.
- **Lazy Loading:** Caricamento asincrono dei moduli (Dashboard, Calendario, Anagrafica) per performance elevate.

- **Servizi Containerizzati:**

- database (PostgreSQL) con volume persistente locale.
- postgres (Backend API).
- frontend (Angular distribuito via Nginx locale).
- revproxy (Nginx principale per esporre la porta 80 e gestire il routing di /api/).
- swagger (Documentazione esposta sulla porta 8081).

- **Variabili d'ambiente:** Gestione centralizzata dei dati sensibili e dei parametri di avvio tramite file `.env`.

Kubernetes: Alta Disponibilità e Storage con Longhorn

La migrazione del cluster a Kubernetes assicura scalabilità orizzontale e auto-healing.

Persistenza con Longhorn

Il database PostgreSQL richiede uno storage persistente ad alte prestazioni. Viene utilizzato **Longhorn** come CSI (Container Storage Interface):

- Storage distribuito e resiliente all'interno del cluster.
- Provisioning dinamico dei volumi tramite la StorageClass longhorn.
- StatefulSet per garantire la persistenza stagna e l'identità del pod database.

Ordinamento dei Manifesti Kubernetes

I manifesti in `k8s/manifests/` sono organizzati alfabeticamente tramite numerazione per un'applicazione fluida in due macro-fasi:

Fase 1: Storage (0-storage/)

- `longhorn.yaml`

Fase 2: Applicazione (1-app/)

- `0-namespace.yaml`
- `1-secrets.yaml`
- `2-database.yaml`
- `3-postgrest.yaml`
- `4-swagger.yaml`
- `5-frontend.yaml`
- `6-revproxy.yaml`
- `7-ingress.yaml`
- `8-cloudflare-tunnel.yaml`

Pubblicazione via Cloudflare Tunnel

Per esporre l'applicazione su internet in sicurezza, si evita il Port Forwarding classico a favore di un **Cloudflare Tunnel**.

- **Funzionamento:** Il pod `ccloudflared` all'interno di Kubernetes stabilisce una connessione sicura in uscita verso i server di Cloudflare.
- **Sicurezza:** Nessuna porta aperta sul firewall di rete locale. Protezione nativa DDoS e TLS automatica.
- **Routing Interno:** Il tunnel instrada il traffico del dominio pubblico `horsecare.sawherd.com` direttamente al servizio interno `revproxy:80` nel cluster.

Risultati Ottenuti

L'applicazione è completamente containerizzata, distribuita in alta disponibilità con storage distribuito (Longhorn), e resa accessibile via web in modo sicuro (Cloudflare Tunnel).

Evoluzioni Future:

- Ampliare e distribuire il cluster k8s per ottenere HA e DR (cluster ibrido?).
- Implementazione di un operatore database avanzato (es. CloudNativePG) per repliche e backup WAL (Azure Flexible PG?).
- Abilitazione di logiche multi-scuderia (Multi-Tenancy) a livello di record.
- Sviluppo di logiche di sincro offline (Offline-First) per l'uso senza connettività di rete.